exercise parking lot control system v2

Last edited by **Peter Høgh Mikkelsen** 3 days ago

Exercise: Parking Lot Control System (PLCS) (V2)

In this exercise, we will implement a Parking Lot Control System that monitors a parking lot and grants access to cars wishing to enter and exit.

Focus

- Thread synchronization with **mutexes** and **condition variables**
- Modeling real-world control logic with threads
- Handling resource constraints (limited parking spaces)

Preparation

- You must have completed the exercise *Thread Synchronization tools*.
- Review the **Park-a-Lot 2000** example from the lecture as inspiration for interaction between cars and guards.
- Only **mutexes and condition variables** may be used in the solution — **semaphores and atomics are not allowed**.

Approval

This exercise must be handed-in. The following requirements must be met:

- Use **mutexes** and **condition variables** (no semaphores).
- Provide working implementations of Activities 1 and 2.
- Provide an attempt for Activity 3 (completion not required).

You must hand-in a PDF on FeedbackFruits containing the following:

- Front page with group number and members plus **link to your gitlab repo**
- Design descriptions, diagrams and answers to questions in the exercise

Your Gitlab repository must contain:

- Code for your solution
- Makefile / CMakeLists.txt to build your project

To have your hand-in approved you must also peer-review a fellow student's submission. You can find the review criteria at the end of this page. Your submission occupies 75% of the grade, the review 25%.

The Park-a-lot 2000

The parking system is sketched in the figure below. Bar gates are located at the entrance and exit.

The entry gate remains closed until an arriving car requests, and is granted access. Upon entry, the entry gate closes. When inside, the car is parked in a slot for a while. The exit gate remains closed until a leaving car requests, and is granted access to leave. When out, the exit gate closes again.

Activity 1: Designing the PLCS

This activity focuses on designing the system for a single car.

Step 1: Sketching the design

- Each **car** is represented by a thread.
- The **entry gate** and **exit gate** are each represented by their own thread.
- Running the program with one car would therefore result in 3 threads.
- There are two ressources that must be guarded: *entry gate* and *exit gate*
- An arriving car requests permission to enter from the *entry gate*.
- An exiting car requests permission to exit from the *exit gate*.
- Gates should remain closed if no car is waiting.

Task

- Create an **activity diagram** that illustrates the flow of each thread and synchronization between them for one car repeatedly entering, staying for a while, and exiting.
- This cycle should repeat until the program is terminated.

Activity 2: Implementing the PLCS

This activity builds on the design and implements it in code.

Step 1: First step - Single car

- Implement your design so that one car can continuously enter, stay, and exit.
- Verify that the behavior matches your design.

Step 2: Multiple cars - The grandiose test

- Extend your implementation to support **multiple cars**.
- Each car should wait a different amount of time inside the parking lot before exiting.
- For now, assume the parking lot has **unlimited capacity**.

Questions

- Did you need to use `std::condition_variable::notify_all()`? Why or why not?
- Cars may not hold the line (they can overtake each other). Why does this happen? Can you think of a way to enforce ordering?

Activity 3: PLCS with Limited Capacity

*This is a challenging extension where the parking lot has a maximum size. Use **time-boxing** for this exercise: do not spend excessive time stuck on it!*

In this extension, the parking lot has a maximum capacity, and cars must wait when the lot is full. In addition to the entry and exit gates, the parking capacity is now treated as a shared resource protected by a mutex.

New Constraints

- The parking lot has a fixed number of spaces.
- Cars entering must check and decrement capacity.
- Cars exiting must increment capacity.
- A mutex must be used to guard access to the capacity variable (to avoid race conditions).
- Cars may need to wait on a condition variable when the lot is full.

Important Change

The **capacity itself** is now a shared resource, protected by a mutex. Cars entering the lot must acquire the entry gate and then lock the capacity mutex and decrease the available capacity before entering. Cars exiting must lock the capacity mutex, increase the available capacity, and acquire the exit gate before leaving.

This means:

- Entering cars acquire locks in the order: **entry gate → capacity mutex**
- Exiting cars acquire locks in the order: **capacity mutex → exit gate**

This ordering is legal but introduces the possibility of circular wait, which you should consider as part of your design reasoning.

Step 1: Design the Signaling Scheme

You must design a signaling approach that ensures:

1. Cars cannot enter when capacity = 0.
2. Cars that arrive when full must wait on a condition variable.
3. When a car exits and increases capacity, waiting cars must be notified.

Things to consider:

- Which condition variable(s) will you use for waiting cars?
- What is the predicate for the wait?
- When is notification sent?
- How is capacity protected by the mutex?

You should sketch the lock acquisition pattern:

- entry gate → capacity mutex
- capacity mutex → exit gate

-and think about whether this ordering could cause circular wait if multiple cars arrive/exit simultaneously.

Hint! One possible tool to manage the parking slots, could be a Monitor.

Step 2: Implement and Test

Implement the capacity-limited PLCS:

- A capacity integer protected by a mutex.
- A condition variable that cars use when the lot is full.
- Proper locking and unlocking of gate and capacity mutexes.
- Cars entering decrement capacity when gaining access.
- Cars exiting increment capacity upon leaving.

Expected Behavior

- Cars stop at the entry gate when the lot is full.
- When a car leaves, one or more waiting cars are notified.
- The parking capacity is never exceeded.

- The system handles multiple cars correctly.

Discussion / Reflection Questions

- Did your design risk circular wait? Why or why not?
- How did lock acquisition order influence deadlock possibility?
- Could a different ordering remove the circular wait risk?
- How would you modify locking strategy to avoid circular wait?

(Completion of this reflection is expected, not necessarily resolving circular wait completely.)